RUNGTA INTERNATIONAL SKILLS UNIVERSITY
BACHELOR OF COMPUTER APPLICATION

BLOOD BANK MANAGEMENT SYSTEM

Python 3 | Console-Based Application

Submitted By	Vaishnavi Jangel
ERP No.	RU-26-01760
Course	BCA AI/ML (Association with Google)
Section	B
Session	2026–27
Submitted To	Satyaprakash Sir

PROJECT REPORT

Project Statement No. 75

1. Project Overview

The Blood Bank Management System is a menu-driven Python application for maintaining blood stock and donor records. It allows a user to add a donor, issue blood units when stock is sufficient, view current stock, and search donor records by blood group. Donor entries are also saved in a text file named **donors.txt**.

Objectives

• Maintain stock for all eight standard blood groups.

• Validate blood group input and require a positive whole-number unit count.

• Prevent issuing more blood units than are available.

• Store donor records in a list of dictionaries and save records to a text file.

• Search donors by blood group and show the matching donor count.

Technologies and Concepts

Technology / Concept	Purpose
Python 3	Main programming language
Dictionary	Maintains blood stock and donor record fields
List	Stores donor records in memory
File handling	Saves donor data to donors.txt
Functions and loops	Organizes operations and repeats the menu
Input validation	Checks blood groups and unit values

Initial Blood Stock

Blood group	A+	A–	B+	B–	AB+	AB–	O+	O–
Units	10	0	5	0	2	0	8	0

2. System Features

• **Add donor:** Accepts a donor name, blood group, and units; updates stock and saves the record.

• **Issue blood:** Issues units only if sufficient stock is available.

• **View stock:** Displays current units for all blood groups.

• **Search donors:** Lists donors matching a blood group and shows the total found.

• **Setup initial data:** Loads the donor records needed for Test Case 4.

• **Exit:** Closes the application.

3. Test Cases and Expected Results

Test case	Input / action	Expected result
TC1: Normal input	Add Anil Verma, B+, 1 unit	Donor saved; B+ stock increases from 5 to 6 units.
TC2: Boundary input	Issue 2 units of AB+	2 units issued; AB+ stock becomes 0.
TC3(a): Insufficient stock	Issue 3 units of A–	Request rejected; 0 units are available.
TC3(b): Invalid group	Add Rina Das, C+, 1 unit	Input rejected; valid blood groups displayed.
TC4: Search donors	Load initial data and search O+	Meena Joshi (1 unit) and Suresh Rao (2 units); total found: 2.

The following output screenshots are included on separate pages for clear viewing and printing.

4. Complete Python Source Code

Source file: one.py. The submitted source code is reproduced below in readable, numbered sections.

```
001  # ========================================
002  # ■ BLOOD BANK MANAGEMENT SYSTEM ■
003  # ========================================
004
005  import os
006
007  # Valid Blood Groups Definition
008  VALID_BLOOD_GROUPS = ["A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-"]
009
010  # Initial Stock Configuration as per problem statement
011  # Note: Initial stock: A+ 10, B+ 5, O+ 8, AB+ 2; all other groups 0.
012  stock = {
013      "A+": 10,
014      "A-": 0,
015      "B+": 5,
016      "B-": 0,
017      "AB+": 2,
018      "AB-": 0,
019      "O+": 8,
020      "O-": 0
021  }
022
023  # Donors Data Structure
024  donors = []
025
026  # ----------------------------------------
027  # ■ FILE HANDLING UTILITIES
028  # ----------------------------------------
029  def save_donor_to_file(name, blood_group, units):
030      """Saves a donor record to donors.txt file"""
031      with open("donors.txt", "a") as f:
032          f.write(f"{name}, {blood_group}, {units}\n")
033
034  def load_initial_donors_for_tc4():
035      """Initializes donors list for Test Case 4 as per requirements"""
036      global donors
037      donors = [
038          {"name": "Meena Joshi", "blood_group": "O+", "units": 1},
039          {"name": "Suresh Rao", "blood_group": "O+", "units": 2}
040      ]
041      with open("donors.txt", "w") as f:
042          f.write("Meena Joshi, O+, 1\n")
043          f.write("Suresh Rao, O+, 2\n")
044
045  # ----------------------------------------
```

4. Complete Python Source Code (continued — lines 46–90)

```
046  # ■■ CORE SYSTEM FUNCTIONS
047  # ----------------------------------------
048  def add_donor():
049      """Add a new donor and update stock & file"""
050      print("\n--- Add Donor ---")
051      name = input("Name: ").strip()
052      blood_group = input("Blood group: ").strip().upper()
053
054      # Validation for Blood Group
055      if blood_group not in VALID_BLOOD_GROUPS:
056          print(f"Invalid blood group. Valid groups: {', '.join(VALID_BLOOD_GROUPS)}")
057          return
058
059      try:
060          units = int(input("Units: ").strip())
061          if units <= 0:
062              print("Units must be a positive integer.")
063              return
064      except ValueError:
065          print("Invalid unit input. Please enter a number.")
066          return
067
068      # Update Stock & Donor List
069      stock[blood_group] += units
070      donors.append({"name": name, "blood_group": blood_group, "units": units})
071      save_donor_to_file(name, blood_group, units)
072
073      # Required Output Format
074      print(f"Donor saved to donors.txt. Stock of {blood_group} is now {stock[blood_group]} units.")
075
076  def issue_blood():
077      """Issue blood units if sufficient stock is available"""
078      print("\n--- Issue Blood ---")
079      blood_group = input("Blood group: ").strip().upper()
080
081      # Validation for Blood Group
082      if blood_group not in VALID_BLOOD_GROUPS:
083          print(f"Invalid blood group. Valid groups: {', '.join(VALID_BLOOD_GROUPS)}")
084          return
085
086      try:
087          units = int(input("Units: ").strip())
088          if units <= 0:
089              print("Units must be a positive integer.")
090              return
```

4. Complete Python Source Code (continued — lines 91–135)

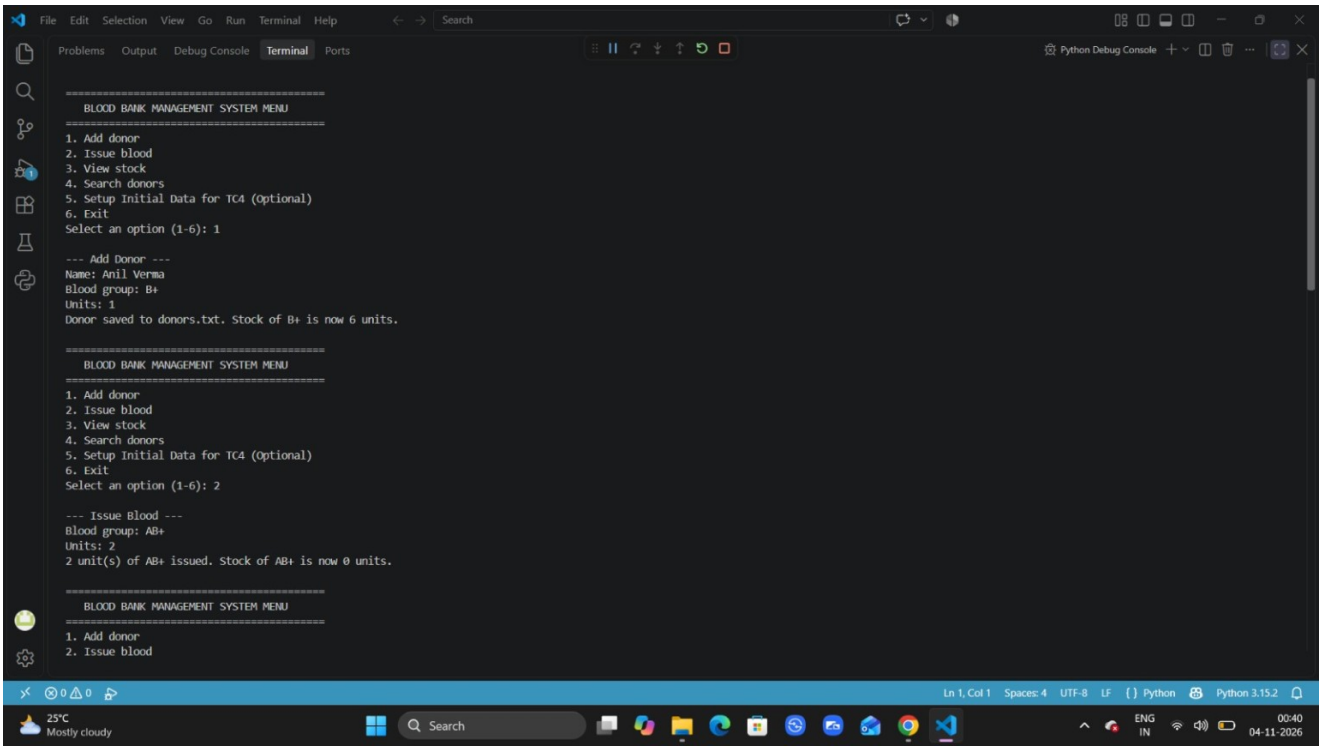
```
091      except ValueError:
092          print("Invalid unit input. Please enter a number.")
093          return
094
095      # Check Available Stock
096      available_stock = stock[blood_group]
097      if units <= available_stock:
098          stock[blood_group] -= units
099          print(f"{units} unit(s) of {blood_group} issued. Stock of {blood_group} is now {stock[
blood_group]} units.")
100      else:
101          print(f"Insufficient stock: only {available_stock} unit(s) of {blood_group} available.")
102
103  def view_stock():
104      """Display current blood stock"""
105      print("\n--- Current Stock ---")
106      for group, count in stock.items():
107          print(f"{group}: {count} units")
108
109  def search_donors():
110      """Search and display donors by blood group"""
111      print("\n--- Search Donors ---")
112      blood_group = input("Blood group: ").strip().upper()
113
114      matching_donors = [d for d in donors if d["blood_group"] == blood_group]
115
116      if matching_donors:
117          print(f"Donors with blood group {blood_group}:")
118          for idx, d in enumerate(matching_donors, 1):
119              print(f"{idx}. {d['name']} - {d['units']} unit(s)")
120          print(f"Total donors found: {len(matching_donors)}")
121      else:
122          print(f"No donors found with blood group {blood_group}.")
123
124  # ----------------------------------------
125  # ■ MAIN APPLICATION MENU
126  # ----------------------------------------
127  def main():
128      while True:
129          print("\n========================================")
130          print("   BLOOD BANK MANAGEMENT SYSTEM MENU")
131          print("========================================")
132          print("1. Add donor")
133          print("2. Issue blood")
134          print("3. View stock")
135          print("4. Search donors")
```

4. Complete Python Source Code (continued — lines 136–159)

```
136         print("5. Setup Initial Data for TC4 (Optional)")
137         print("6. Exit")
138
139         choice = input("Select an option (1-6): ").strip()
140
141         if choice == '1':
142             add_donor()
143         elif choice == '2':
144             issue_blood()
145         elif choice == '3':
146             view_stock()
147         elif choice == '4':
148             search_donors()
149         elif choice == '5':
150             load_initial_donors_for_tc4()
151             print("Initial donor data for Test Case 4 loaded successfully!")
152         elif choice == '6':
153             print("Exiting application. Thank you!")
154             break
155         else:
156             print("Invalid choice! Please select between 1 and 6.")
157
158 if __name__ == "__main__":
159     main()
```

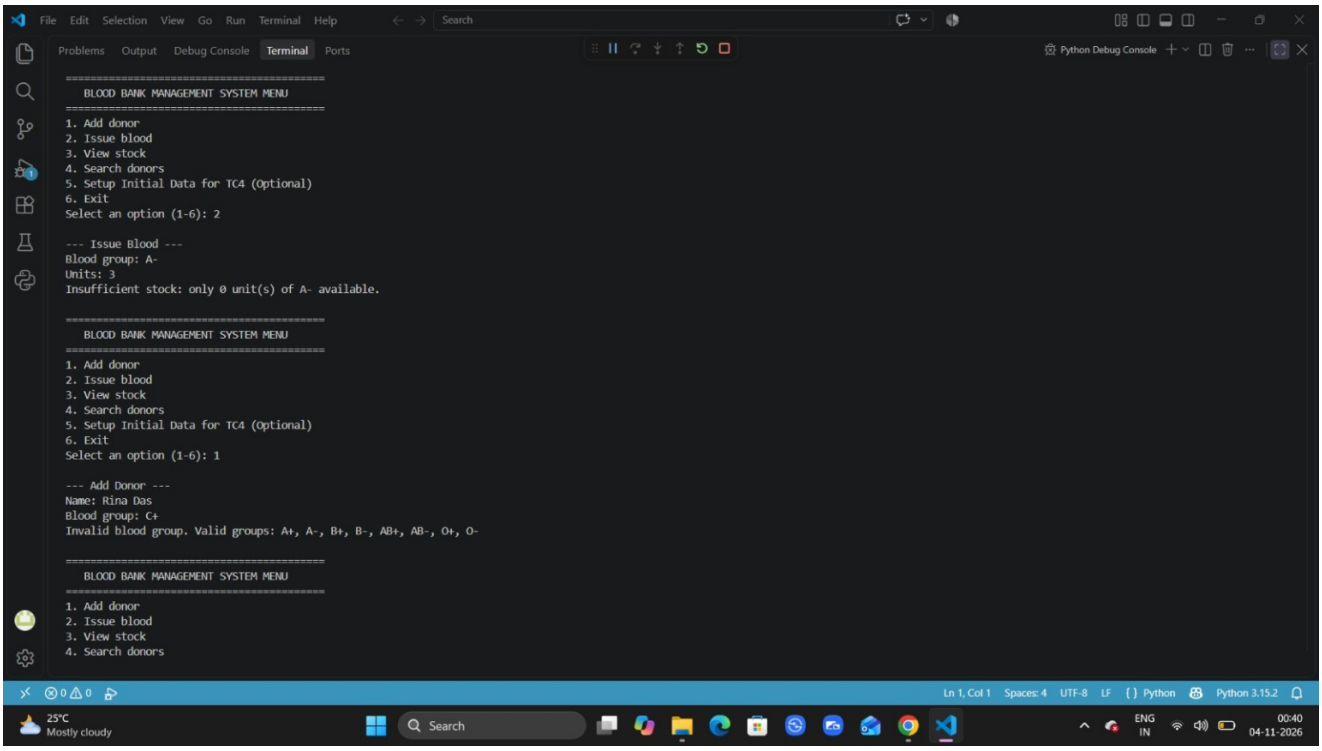
5. Program Output Screenshots

Output Screenshot 1 — Add Donor and Issue Blood



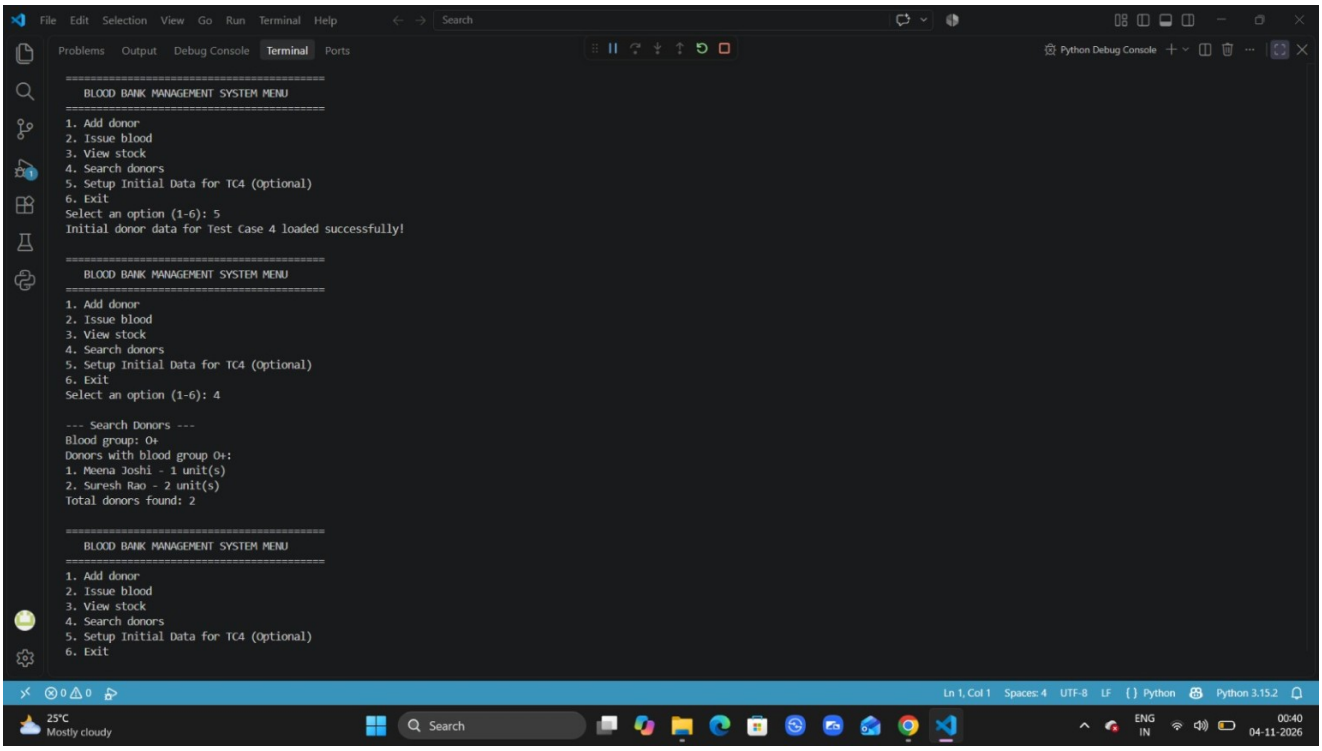
5. Program Output Screenshots

Output Screenshot 2 — Insufficient Stock and Invalid Blood Group



5. Program Output Screenshots

Output Screenshot 3 — Initial Data and Donor Search



6. Conclusion

The Blood Bank Management System demonstrates core Python programming concepts through a practical, menu-driven application. It maintains blood stock, validates input, saves donor information to a text file, and searches donor records by blood group. The included screenshots document key test scenarios and their console outputs.

How to Run the Program

• Keep the Python source file saved as one.py.

• Open a terminal in the folder containing the file.

• Run: **`python one.py`**

• Choose a menu option from 1 to 6 and follow the prompts.

Submitted by: Vaishnavi Jangel

Submitted to: Satyaprakash Sir